
CSE D 501

Clustering cont'd, Fairness

Taylor Kessler Faulkner

University of Washington

November 5, 2025

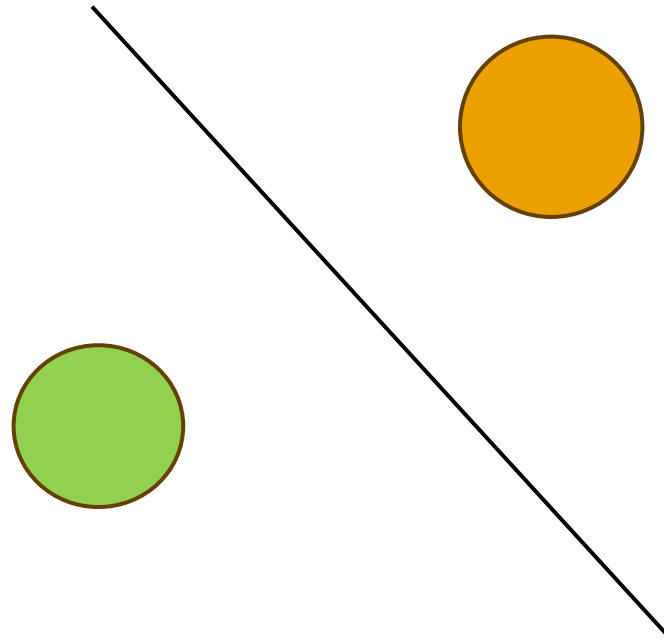


Admin

- Exam releasing on Friday!
 - While the exam is out, we will have modified policies for office hours and the message board; we will answer clarifying or logistical questions, but will not help with specific questions on the exam or give hints.
 - Only written questions, no code
 - Not a timed exam; can take over multiple days
 - Due on **Tuesday, 11/11 by 11:59 PM**
 - **No late days on exams**
- Registration for winter course is open!
 - You can enroll in CSE D 502 (CV and Deep Learning) using SLN code 13149
 - More information in message from Kate on Modern AI Methods Students Ed board

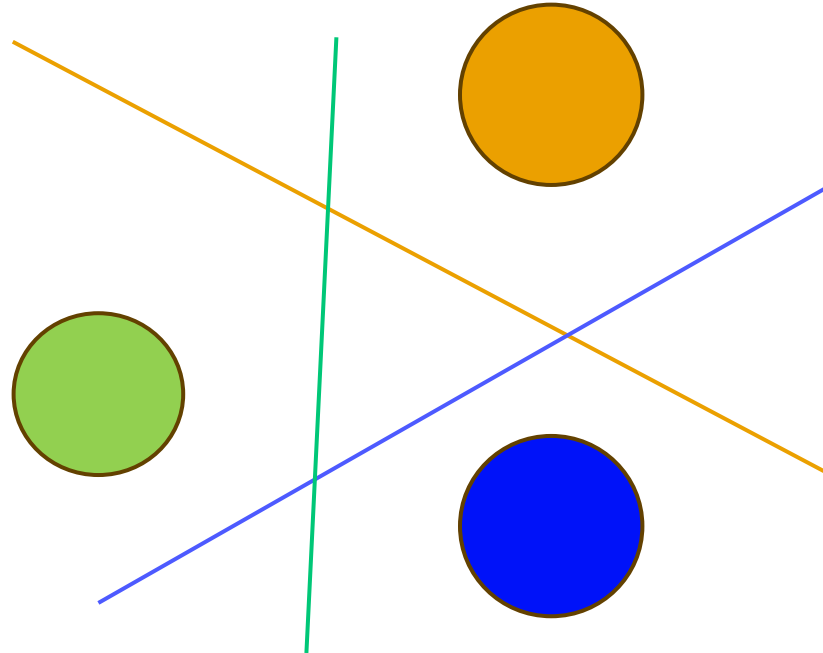
Clarifications

- Clarification on linear separability
 - Question that came up: what does linear separability look like for more than 2 classes?
 - For two classes, we know that you just need to be able to draw a line (or hyperplane in multiple dimensions) between the classes



Clarifications

- Clarification on linear separability
 - Question that came up: what does linear separability look like for more than 2 classes?
 - For two classes, we know that you just need to be able to draw a line (or hyperplane in multiple dimensions) between the classes
 - What about more than 2?
 - This is still linearly separable: we can draw a line that separates any one class from the other two



Today's Class

- Clustering continued (GMMs, DBSCAN)
- Break
- Fairness definitions
- Break
- Discussion

Hard Clustering

- K-means associates each point with one (and only one) cluster
- What if we want probabilities of each point belonging to all of the clusters, like with logistic regression?
- We can use a different algorithm, Gaussian Mixture Models (GMMs)

Gaussian Mixture Model

- Gaussian Mixture Models (GMMs) take a probabilistic approach to clustering
- A Gaussian Mixture is a function composed of several Gaussians (in our case, K)
- Each Gaussian is comprised of:
 - A mean μ (the center)
 - A covariance matrix Σ
 - In 1D, this is just the variance/width of the Gaussian
 - With more features, it is a square matrix where the diagonal elements are the variance/width of the gaussian for each feature, and the other elements are the correlation/covariance between pairs of features
 - A mixing probability π (the probability that a given data point was generated from this Gaussian, effectively the “size” of the Gaussian)
 - The mixing probabilities for all Gaussians must sum to 1: $\sum_{i=1}^K \pi_i = 1$

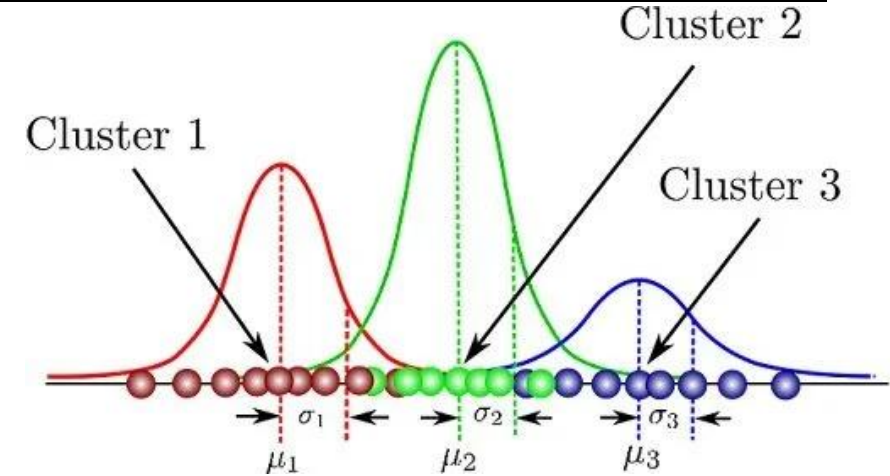


Image from <https://builtin.com/articles/gaussian-mixture-model>

GMM Demo

Gaussian Mixture Model

- Gaussian in 3D:

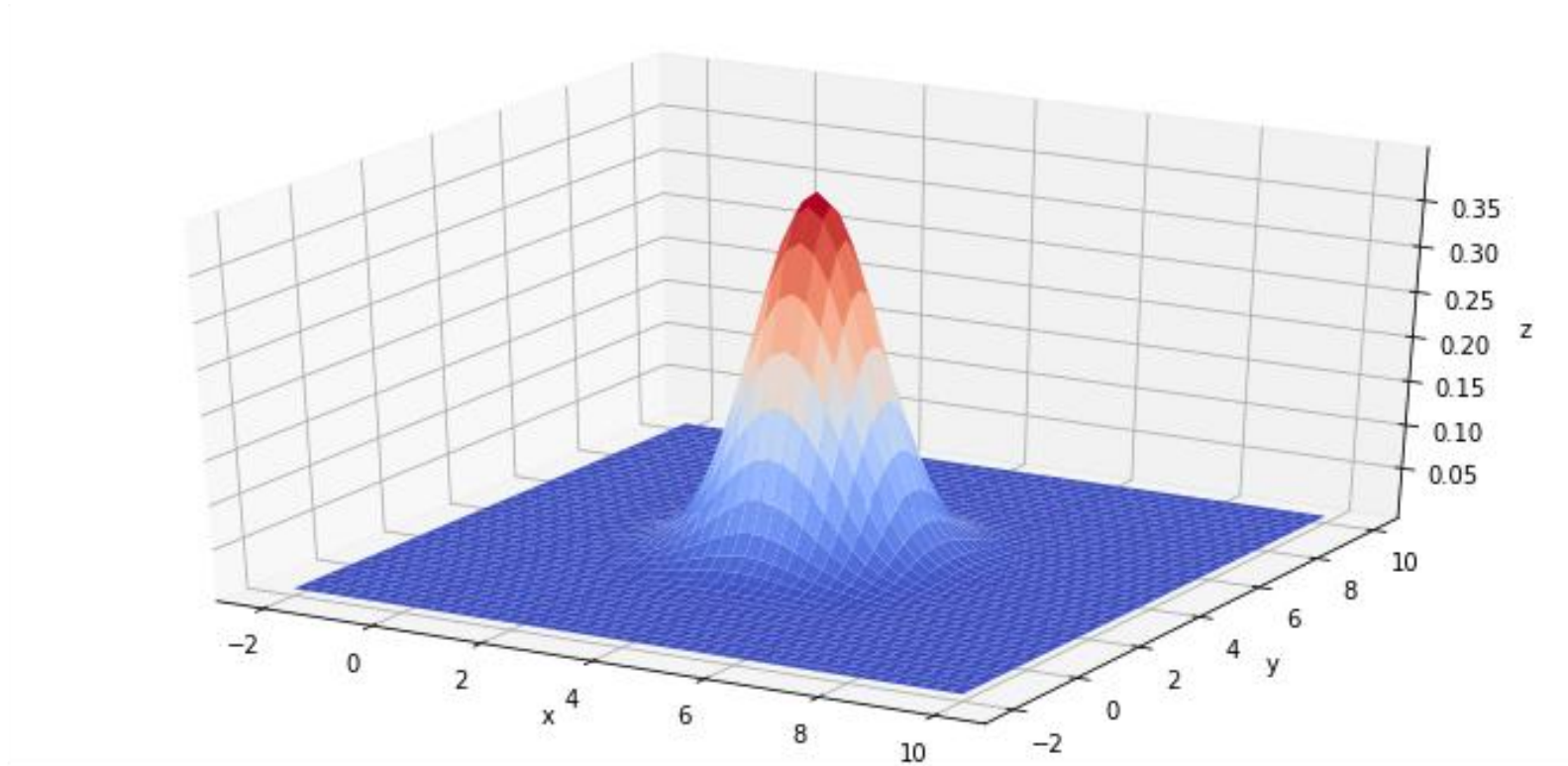


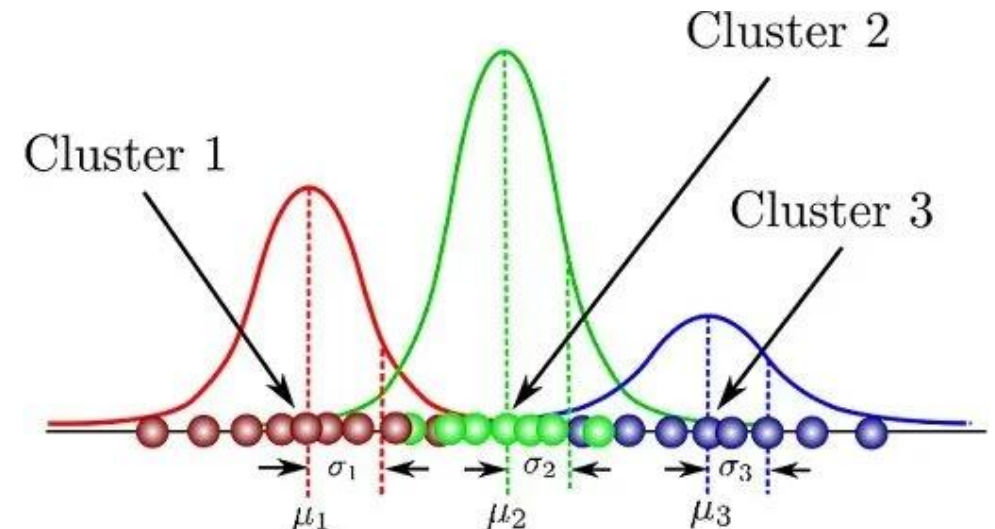
Image from <https://medium.com/data-science/a-python-tutorial-on-generating-and-plotting-a-3d-gaussian-distribution-8c6ec6c41d03>

Gaussian Mixture Model

- We want to define a set of K Gaussians that maximize the likelihood of our dataset
 - Find K means, covariances matrices, and mixing probabilities
- The Gaussian probability density function is given by the following equation:

$$\mathcal{N}(x|\mu, \Sigma) = \frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu)\right)$$

- Where D is the number of features
- μ is the mean
- Σ is the covariance matrix
- π is just pi here (3.14), not the mixing coefficient



Gaussian Mixture Model

- We want to define a set of K Gaussians that maximize the likelihood of our dataset
 - Find K means, covariances matrices, and mixing probabilities
- Let $z_k[n]$ be a variable that is 1 when x_n comes from Gaussian k
 - z_k is a vector of length N
- We want to find $P(z_k[n] = 1|x_n)$: given a datapoint x_n , what is the probability it came from Gaussian k ?

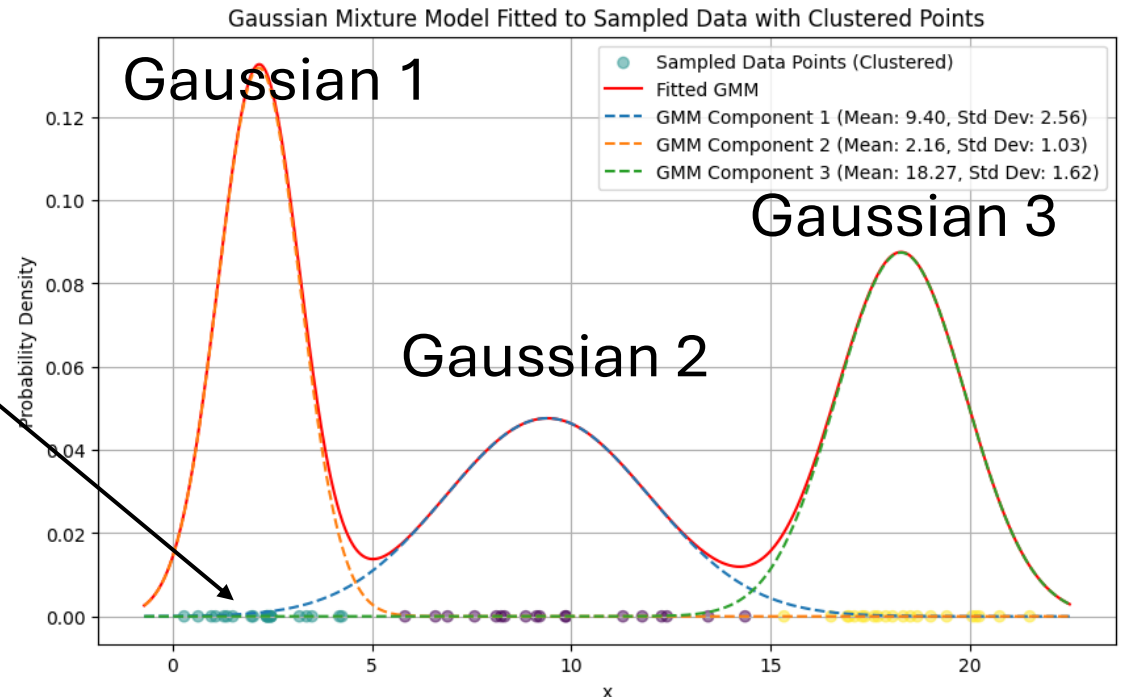
$$P(z_1[5] = 1|x_5)$$

We want this probability to be high!

$$P(z_2[5] = 1|x_5)$$

$$P(z_3[5] = 1|x_5)$$

We want these probabilities to be low

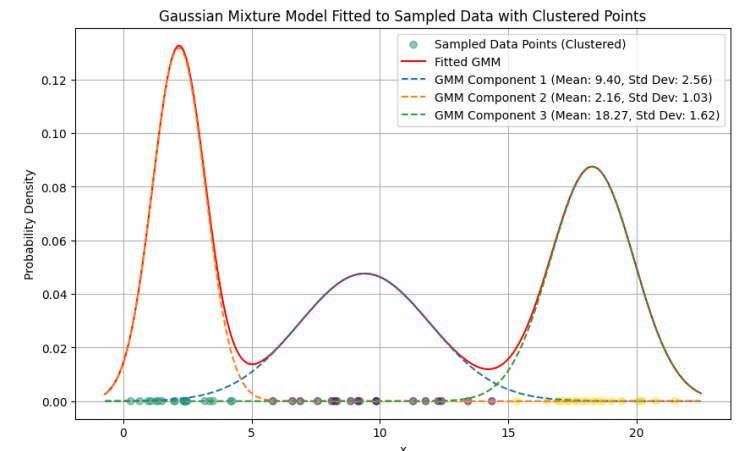


Gaussian Mixture Model

- Some helpful pieces of information:
 - $P(z_k[n] = 1) = \pi_k$: the probability of observing a datapoint from Gaussian k is the mixing coefficient π_k
 - Aka for any x_n , the probability that it came from Gaussian k is π_k , because $\pi_k * N$ datapoints are assumed to come from Gaussian k
 - $P(x_n | z_k[n] = 1) = \mathcal{N}(x_n | \mu_k, \Sigma_k)$: the probability of observing datapoint x_n given that it came from Gaussian k is just the Gaussian function
 - Aka we're assuming that the datapoints were generated from each defined Gaussian

Example $z_k =$

$[0,0,0,1,1,0,1,0,1,0,1,1,0,0,0,0,\dots 0,0,0]$



Gaussian Mixture Model

- Using the product rule of probabilities ($P(A, B) = P(A|B)P(B)$), we can find that $P(\mathbf{x}_n)$ (the probability of seeing a datapoint $\mathbf{x}_n$ in our dataset) is

$$P(\mathbf{x}_n) = \sum_{k=1}^K P(\mathbf{x}_n, z_k[n]) = \sum_{k=1}^K P(\mathbf{x}_n | z_k[n]) P(z_k[n] = 1) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$$

- The likelihood of the entire dataset $P(\mathbf{X})$ can now be defined as the joint probability of all of our datapoints (probability of x_1 and x_2 and ... x_N)

$$P(\mathbf{X}) = \prod_{n=1}^N P(\mathbf{x}_n) = \prod_{n=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$$

- Just like in Logistic Regression, we want to convert products to summations for easier computation, so we'll use log-likelihood

$$\ln(P(\mathbf{X})) = \sum_{n=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k) \right)$$

Gaussian Mixture Model

- Unfortunately, we can't apply **MLE** (Maximum Likelihood Estimation) like we did for logistic regression
 - The function is complex, non-linear, and the mixture weights are not observable from the data
- Fortunately, we can use the **Expectation-Maximization** (EM) algorithm instead!

Expectation-Maximization (EM)

- We start by randomly initializing model parameters
 - Can be done by randomly choosing k centroids (or running k-means to choose initial centroids), and then computing GMM parameters based on centroids
- Then repeat the Expectation and Maximization steps until convergence:
 - Expectation: Compute the expected log-likelihood of the model
 - Maximization: Update the parameters of the model to maximize the log-likelihood of the observed data
- Convergence: often a maximum number of iterations or when the change in log-likelihood is minimal (under some ϵ)

Gaussian Mixture Model

- How can we calculate the probability of each z_k **given** X
- We know the probability of seeing any $\mathbf{x}_n$

$$P(\mathbf{x}_n) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$$

- The probability of seeing $\mathbf{x}_n$ given z_k

$$P(\mathbf{x}_n | z_k[n] = 1) = \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$$

- And the probability of observing a datapoint from Gaussian z_k

$$P(z_k[n] = 1) = \pi_k$$

- What we really want to know is the probability of each $z_k[n]$ **given** $\mathbf{x}_n$
 - Aka, given some $\mathbf{x}_n$, what is the probability that $z_k[n] = 1$? Where an example $z_k = [0,0,0,1,1,0,1,0,1,0,1,1,0,0,0,0,\dots,0,0,0]$

Probability Aside: Bayes' Rule

- When we have two events, A and B , we might want to calculate the probability of A happening given that B has happened.
- Example: consider a rare disease with a test that can have false positives and negatives. The disease is present in 1% of the population. If someone has the disease, the test will give a **true positive** result 99% of the time. The overall probability of testing positive (true positives and false positives) is 2%.
- Let A be the event in which the person has the disease, and B be a positive test result.
- So $P(A) = 0.01$, $P(B|A) = 0.99$, and $P(B) = 0.02$
- Question: what is the probability that a person has the disease given that they test positive?
 - Or, what is $P(A|B)$?

Probability Aside: Bayes' Rule

- Question: what is the probability that a person has the disease given that they test positive?
 - Or, what is $P(A|B)$?
- $P(A) = 0.01$, $P(B|A) = 0.99$, and $P(B) = 0.02$
- Bayes' rule says

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- So even though we don't know $P(A|B)$, since we know $P(B|A)$ we can calculate it!
- $P(A|B) = \frac{0.99*0.01}{0.02} = 0.495$

Gaussian Mixture Model

- Bayes' rule

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- We know

- $P(\mathbf{x}_n) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$
- $P(\mathbf{x}_n | z_k[n] = 1) = \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$
- $P(z_k[n] = 1) = \pi_k$

- So to calculate $P(z_k = 1 | \mathbf{x}_n)$:

$$P(z_k[n] = 1 | \mathbf{x}_n) = \frac{P(\mathbf{x}_n | z_k[n] = 1)P(z_k[n] = 1)}{P(\mathbf{x}_n)} = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}$$

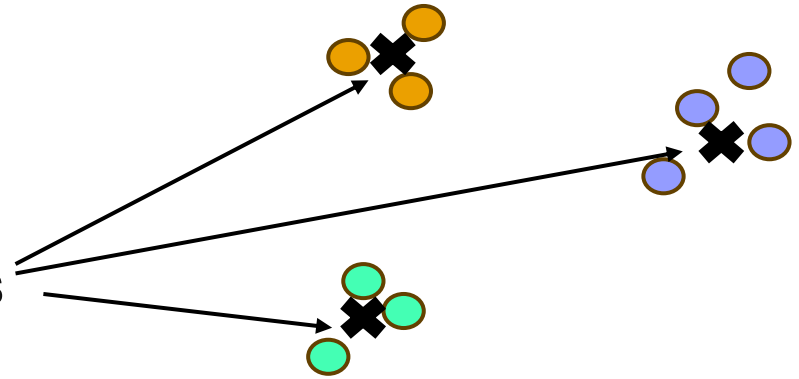
- Let $\gamma(z_k[n]) = P(z_k[n] = 1 | \mathbf{x}_n) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}$

Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 1: initialize θ
 - Often done by running K-means
- GMM initial means: K-means cluster centroids
- GMM initial covariances: covariance matrices of data points in each K-means cluster i (containing N_i datapoints)
 - $$\Sigma[i] = \frac{1}{N_i - 1} \sum_{j=1}^{N_i} (x_j - \mu_i)(x_j - \mu_i)^T$$
- Mixing probabilities: for each K-means cluster i , calculate $\frac{N_i}{N}$

Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 1: initialize θ
 - Often done by running K-means
- GMM initial means: K-means cluster centroids
- GMM initial covariances: covariance matrices of data points in each K-means cluster i (containing N_i datapoints)

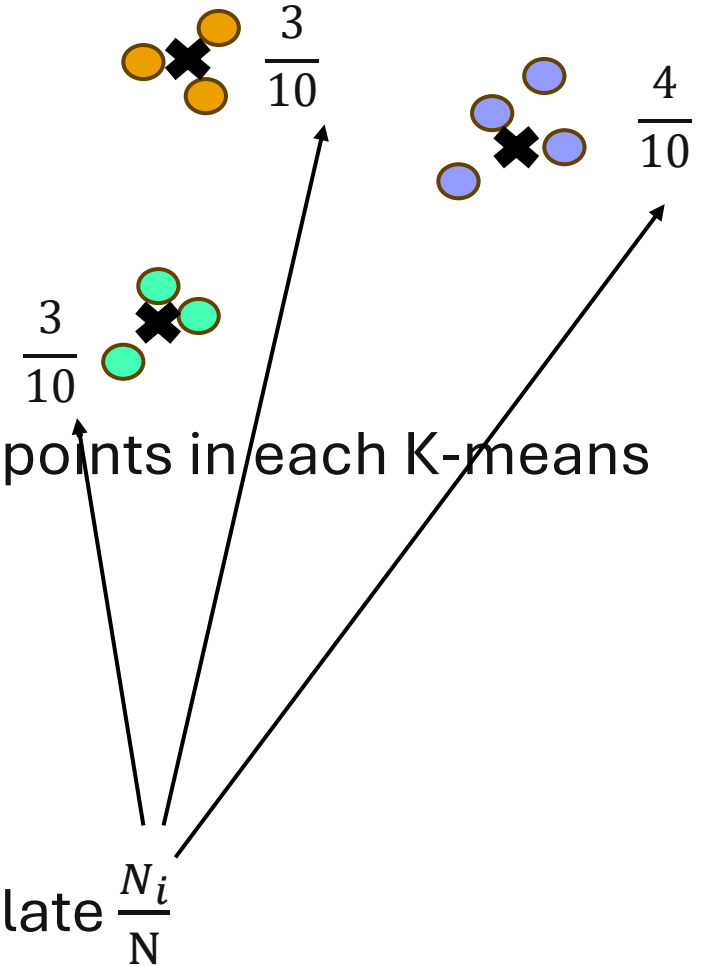


- $$\Sigma[i] = \frac{1}{N_i - 1} \sum_{j=1}^{N_i} (x_j - \mu_i)(x_j - \mu_i)^T$$

- Mixing probabilities: for each K-means cluster i , calculate $\frac{N_i}{N}$

Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 1: initialize θ
 - Often done by running K-means
 - GMM initial means: K-means cluster centroids
 - GMM initial covariances: covariance matrices of data points in each K-means cluster i (containing N_i datapoints)
 - $\Sigma[i] = \frac{1}{N_i - 1} \sum_{j=1}^{N_i} (x_j - \mu_i)(x_j - \mu_i)^T$
 - Mixing probabilities: for each K-means cluster i , calculate $\frac{N_i}{N}$



Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 2: Expectation step
 - We want to evaluate $Q(\theta^*, \theta) = \mathbb{E}[\ln P(X, Z | \theta^*)] = \sum_Z P(Z | X, \theta) \ln(P(X, Z | \theta^*))$
 - $\mathbb{E}$: expected value (what we "expect" this value to be; the weighted average of values * probabilities)
 - Here: weighted average of the log likelihood of our data X and indicator variables Z given the optimal GMM parameters θ^*
 - Weighted by the probability of each Z given the data and current GMM parameters θ
 - Summed over all possible Z s

Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 2: Expectation step
 - We want to evaluate $Q(\theta^*, \theta) = \mathbb{E}[\ln P(X, Z | \theta^*)] = \sum_Z P(Z | X, \theta) \ln(P(X, Z | \theta^*))$
 - We know $P(Z | X, \theta)$: $P(z_k = 1 | \mathbf{x}_n) = \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)}{\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)} = \gamma(z_{nk})$
 - What about $P(X, Z | \theta^*)$?
 - This is the **complete likelihood of the model, including both X and Z**
 - Definition: $P(X, Z | \theta^*) = \prod_{n=1}^N \prod_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)$
 - Product over all K Gaussians and N datapoints of the probability of all datapoints under each Gaussian component
 - Once again, we want to work with log-likelihood and summations
 - $\log(P(X, Z | \theta^*)) = \log(\prod_{n=1}^N \prod_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k)) = \sum_{n=1}^N \sum_{k=1}^K [\log(\pi_k) + \log(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))]$
 - Now we can replace $P(Z | X, \theta)$ and $P(X, Z | \theta^*)$ in $Q(\theta^*, \theta)$
 - $Q(\theta^*, \theta) = \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) [\log(\pi_k) + \log(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))]$

Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 3: Maximization step
 - We want to find $\theta^* = \operatorname{argmax}_{\theta} Q(\theta^*, \theta)$
 - This is an optimization problem! It also has the constraint that $\sum_{i=1}^K \pi_i = 1$
 - Mixing probabilities must sum to 1
 - Question: where have we seen an optimization problem with constraints before?

Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 3: Maximization step
 - We want to find $\theta^* = \operatorname{argmax}_{\theta} Q(\theta^*, \theta)$
 - This is an optimization problem! It also has the constraint that $\sum_{i=1}^K \pi_i = 1$
 - Mixing probabilities must sum to 1
 - Question: where have we seen an optimization problem with constraints before?
 - Answer: SVMs! We're going to use Lagrange multipliers again to transform to just an optimization problem
 - $Q(\theta^*, \theta) = \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) [\log(\pi_k) + \log(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))] - \lambda(\sum_{k=1}^K \pi_k - 1)$
 - And now we have an equation we can apply maximum likelihood to!

Expectation Maximization

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 3: Maximization step
 - $Q(\theta^*, \theta) = \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) [\log(\pi_k) + \log(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))] - \lambda(\sum_{k=1}^K \pi_k - 1)$
 - If we take the derivatives of $Q(\theta^*, \theta)$ with respect to π , μ , and Σ , and set them to 0, we find that:
 - $$\pi_k = \frac{\sum_{n=1}^N \gamma(z_{nk})}{N}$$
 - $$\mu_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n}{\sum_{n=1}^N \gamma(z_{nk})}$$
 - $$\Sigma_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T}{\sum_{n=1}^N \gamma(z_{nk})}$$
 - These calculations give us the next parameters to try!

Expectation Maximization Overview

- Let θ be the GMM parameters (means, covariances, and mixing probabilities)
- Step 1: initialize θ
- Until convergence:
 - Step 2: Expectation
 - Calculate $Q(\theta^*, \theta) = \sum_{n=1}^N \sum_{k=1}^K \gamma(z_{nk}) [\log(\pi_k) + \log(\mathcal{N}(\mathbf{x}_n | \mu_k, \Sigma_k))]$
 - Step 3: Maximization
 - Calculate $\theta = [\pi_k, \mu_k, \Sigma_k]$
 - $\pi_k = \frac{\sum_{n=1}^N \gamma(z_{nk})}{N}$
 - $\mu_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n}{\sum_{n=1}^N \gamma(z_{nk})}$
 - $\Sigma_k = \frac{\sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \mu_k)(\mathbf{x}_n - \mu_k)^T}{\sum_{n=1}^N \gamma(z_{nk})}$

Finding best k for GMM

- For models that maximize likelihood, the BIC (Bayes Information Criterion) is often used
- $BIC = -2\log(L) - \frac{K}{2}\log(N)$
 - L is the log-likelihood of the data
 - K is the number of clusters
 - N is the number of samples
- Can use elbow method, just like WSS for K-means

Gaussian Mixture Model

- Pros and Cons of GMMs
 - Pros
 - Can handle elliptical and irregular cluster shapes
 - Can handle clusters of varying sizes
 - Can give probabilities, and allows for overlapping clusters
 - Cons
 - High computational cost
 - Sensitive to initialization
 - Can be mitigated through **restarts**, just like K-means
 - Assumes data is a mixture of Gaussians

Break

Get some water, stretch your legs, chat, be back in 10

DBSCAN

- What if we want to detect outliers?
- With our current clustering algorithms, all points are assigned to a cluster
- You could select less populated clusters, clusters with low density (spread out points), or label datapoints with a “large” distance to their cluster centroids outliers
- Or, we can use a clustering algorithm built to detect outliers, like DBSCAN

DBSCAN

- DBSCAN categorizes datapoints into three types
 - **Core points:** these points have a sufficient number of neighbors within a specified radius
 - Radius is defined by hyperparameter ϵ
 - Sufficient number of neighbors defined by hyperparameter MinPts
 - **Border points:** these points are near core points, but do not have enough neighbors to be core points
 - **Noise points:** these points do not belong to a cluster

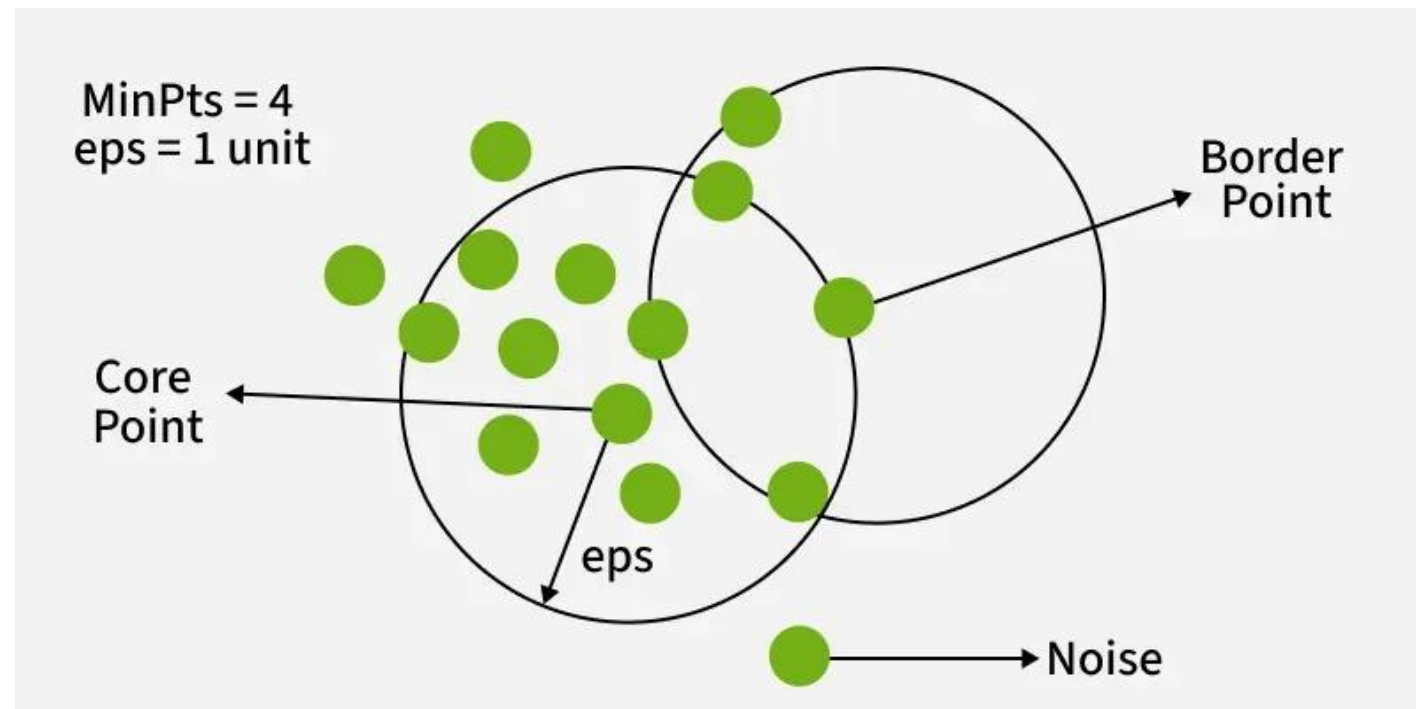
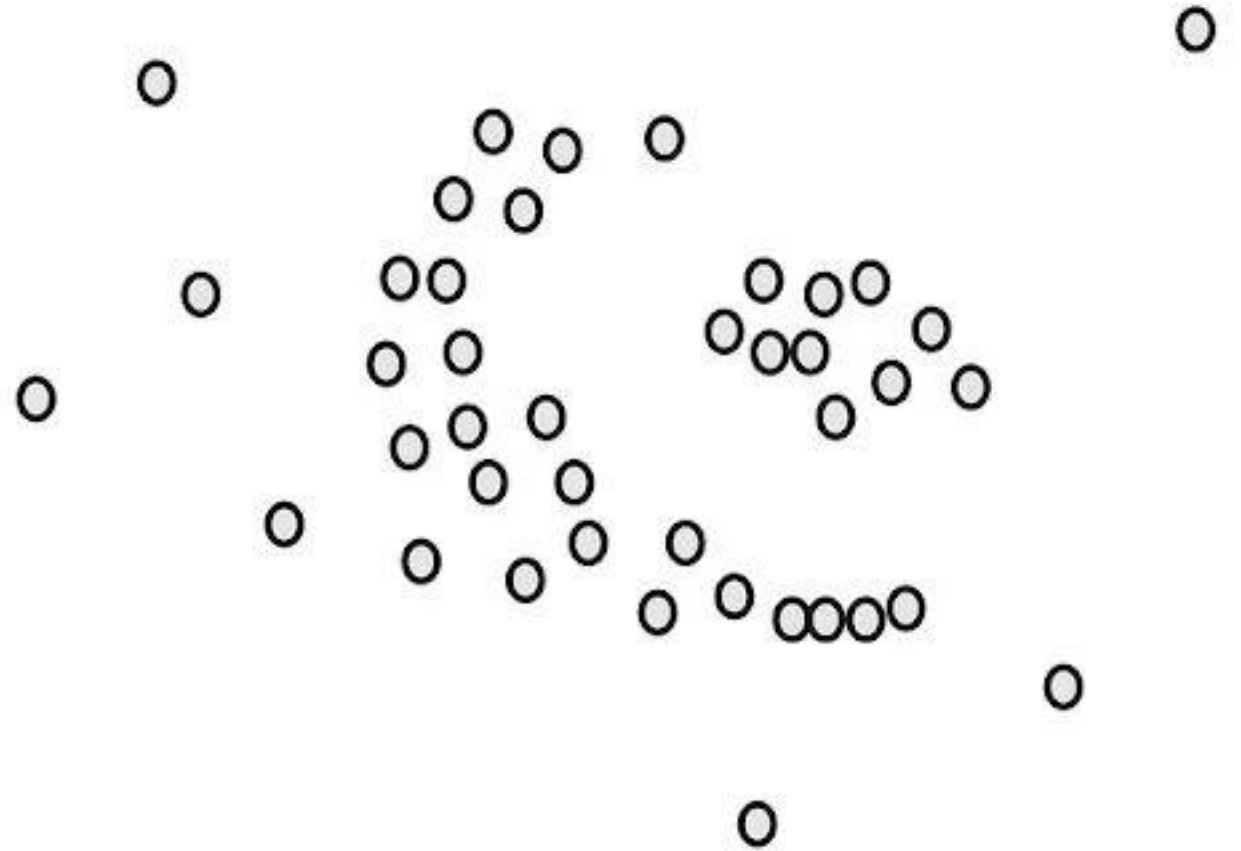


Image from
<https://www.geeksforgeeks.org/machine-learning/dbscan-clustering-in-ml-density-based-clustering/>

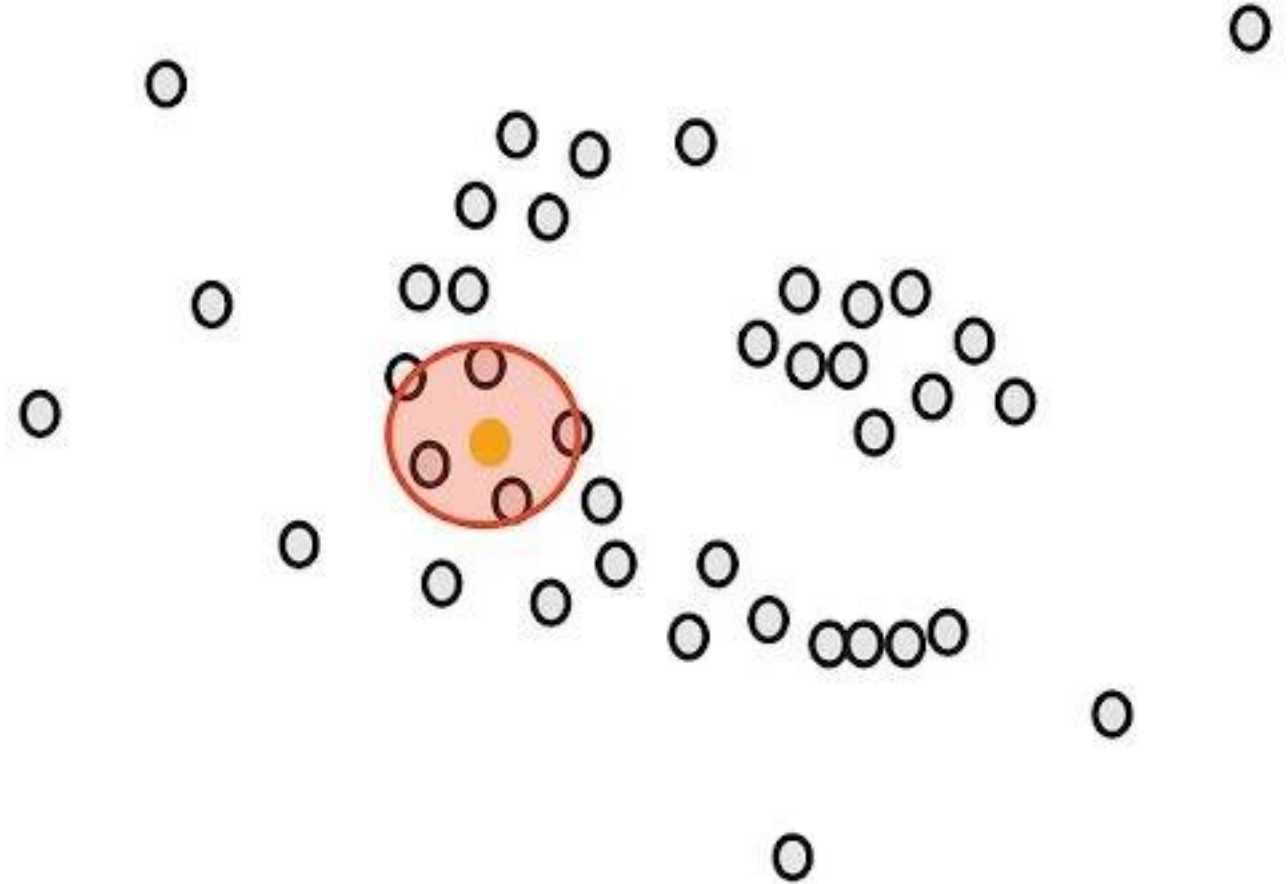
DBSCAN

- DBSCAN algorithm



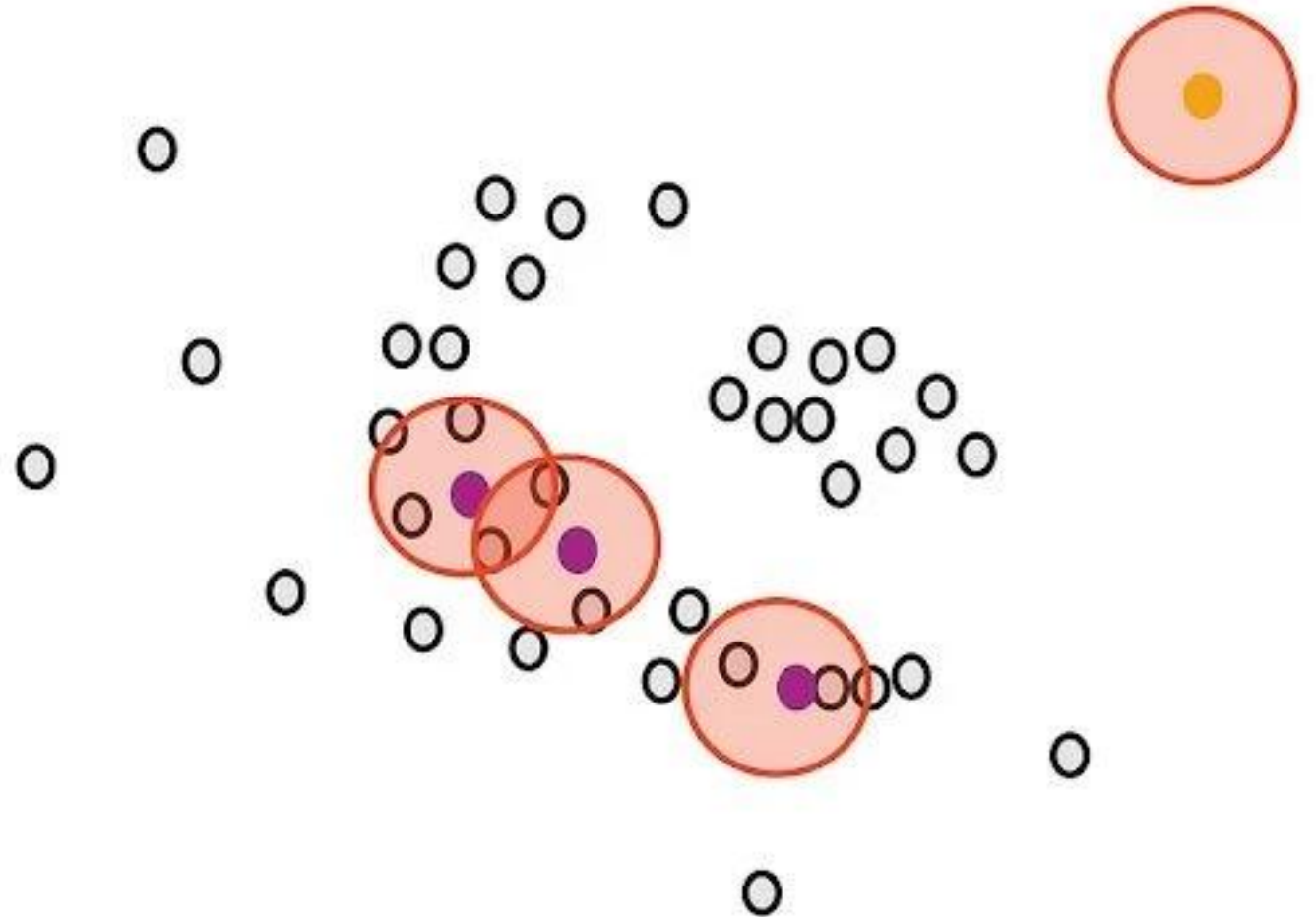
DBSCAN

- DBSCAN algorithm
- Start with a random point, examine the circle of radius ϵ around point



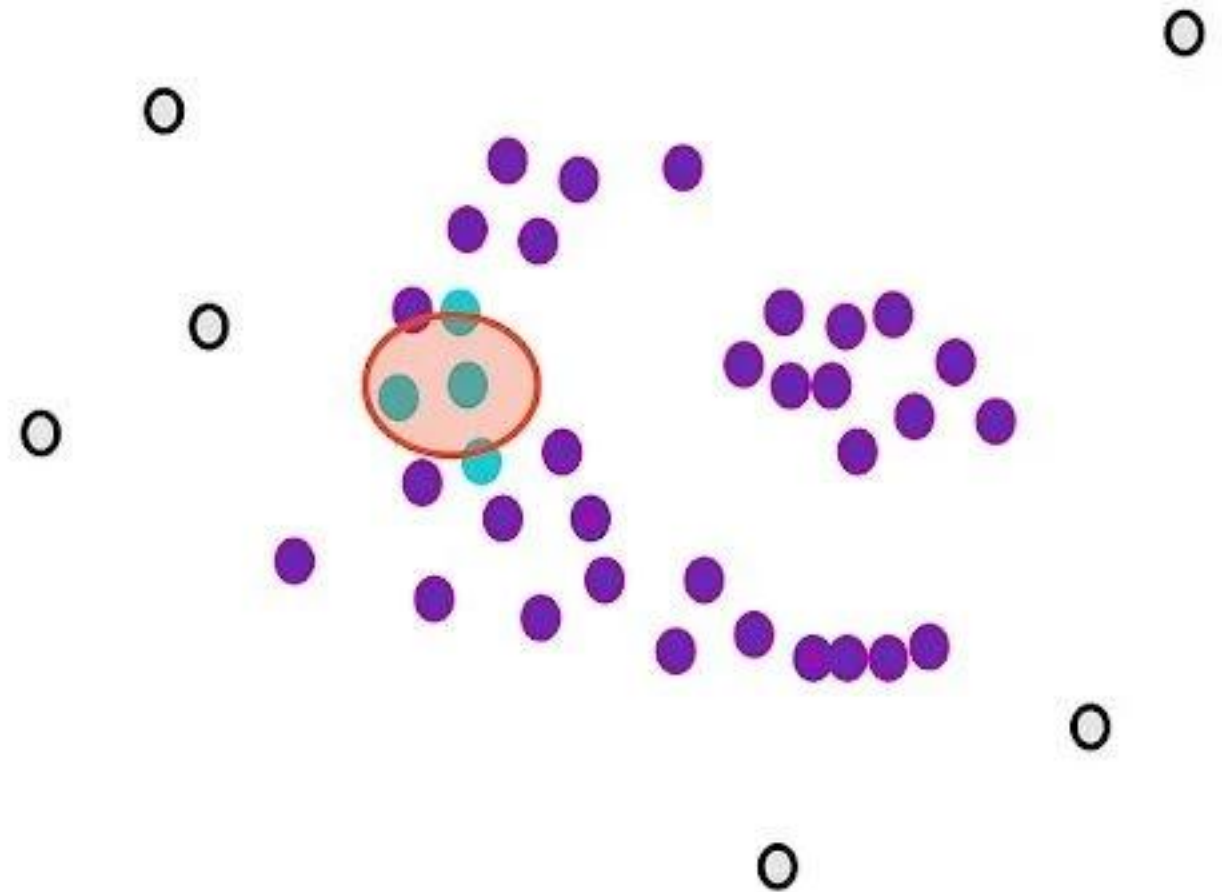
DBSCAN

- DBSCAN algorithm
- Start with a random point, examine the circle of radius ϵ around point
 - If there are at least MinPts datapoints within this circle, the point is labeled a **core point**



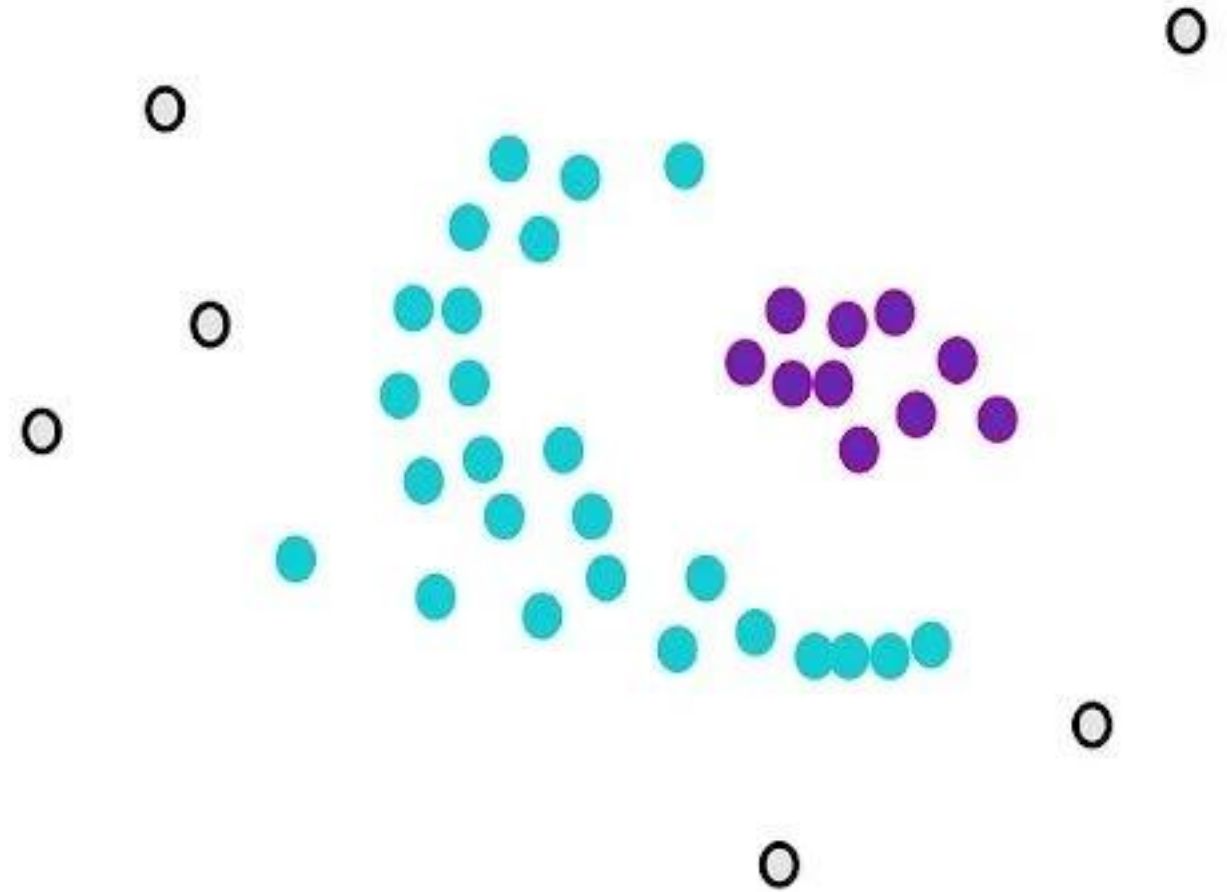
DBSCAN

- DBSCAN algorithm
- Start with a random point, examine the circle of radius ϵ around point
 - If there are at least MinPts datapoints within this circle, the point is labeled a **core point**
- Randomly select a core point and assign it as the first point in our first cluster
 - Any points within ϵ are also added



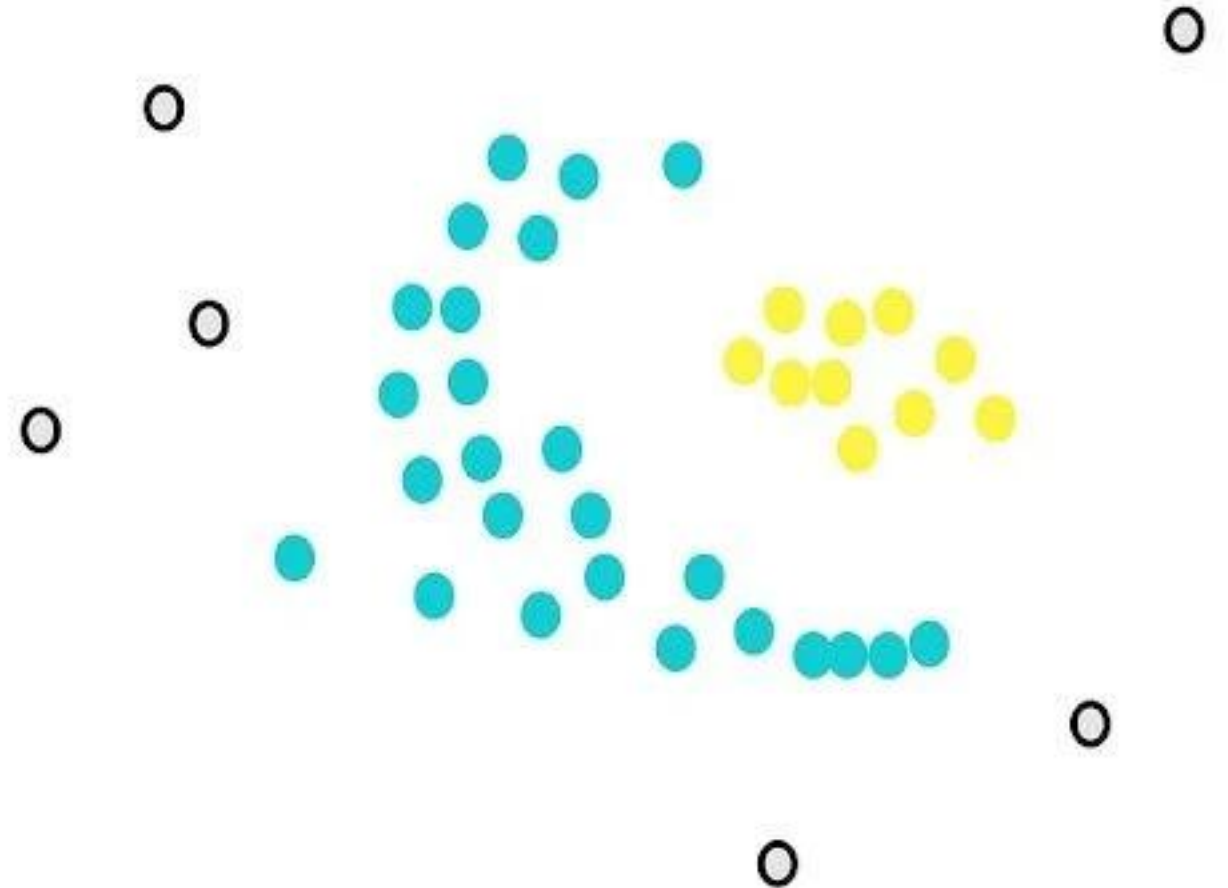
DBSCAN

- DBSCAN algorithm
- Start with a random point, examine the circle of radius ϵ around point
 - If there are at least MinPts datapoints within this circle, the point is labeled a **core point**
- Randomly select a core point and assign it as the first point in our first cluster
 - Any points within ϵ are also added
 - For each of these neighbors
 - If neighbor is a core point, add neighbors to cluster (and so on)
 - If neighbor is not a core point, mark as **border point**



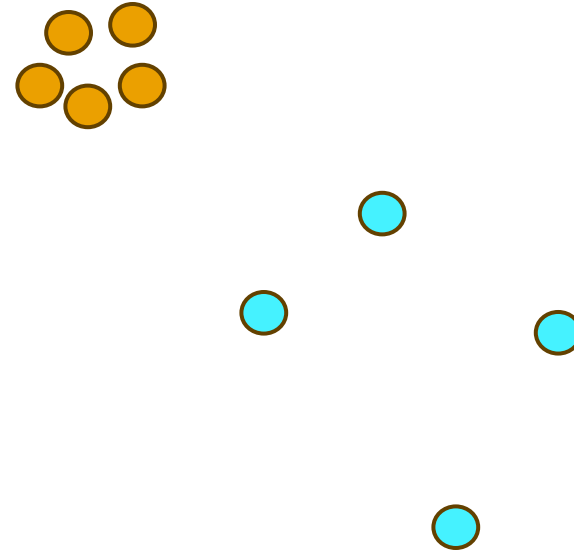
DBSCAN

- DBSCAN algorithm
- Start with a random point, examine the circle of radius ϵ around point
 - If there are at least MinPts datapoints within this circle, the point is labeled a **core point**
- Randomly select a core point and assign it as the first point in our first cluster (**repeat until no more cores**)
 - Any points within ϵ are also added
 - For each of these neighbors
 - If neighbor is a core point, add neighbors to cluster (and so on)
 - If neighbor is not a core point, mark as **border point**



DBSCAN notes

- DBSCAN has difficulty finding clusters of **varying densities**
- This is due to the ϵ parameter defining how close points need to be to be included in any cluster!
- Also, non-core points will be assigned to the first cluster that meets the requirement of closeness
 - So not deterministic, since core points are chosen at random



Choosing DBSCAN hyperparameters

- K-Nearest Neighbors (KNN) distance plot
- Determine MinPts
 - Usually determined by dimensionality of data $\max(3, 2 * D - 1)$
- Calculate the distance to the k-th nearest neighbor, where $k = MinPts$ for every datapoint
- Sort these distances from small to large and plot them
- Find the elbow point; the corresponding distance is a good estimate for ϵ

Clustering Algorithms Compared

- Agglomerative clustering
 - No need to choose k beforehand
 - Provides a visualization of cluster relationships (dendrogram)
 - Complex cluster shapes
 - Computationally expensive
 - Cutoff point can be subjective
 - Greedy algorithm
- DBSCAN
 - No need to choose k
 - Clusters can take any shape
 - Good for outlier detection
 - Can struggle with differing densities of clusters
 - Can struggle with high-dimensional data
- K-means
 - Easy to implement
 - Efficient
 - Sensitive to initial centroids
 - Have to choose k in advance
 - Only same-size spherical clusters
- GMMs
 - Provides probability estimates
 - Computationally more expensive than k-means (less than hierarchical)
 - Have to choose k in advance
 - Sensitive to initial parameters
 - Assumes Gaussian distributions

Topic shift: Fairness in ML

Topic shift: Fairness in ML

- We've talked about bias in datasets
 - Different sources of bias (historical, measurement, etc) can cause our models to encode bias
- How can we determine whether our ML models are making fair decisions?
- First question: what does it mean to be **fair**?
- Difficult question with no single answer
- We have some mathematical definitions that can provide insight
- For now, we'll specifically talk in the context of classification

Fairness in ML

- We'll use an example of hiring decisions
- This example will make many simplifying and unrealistic assumptions, as many models do
- In our scenario:
 - There is a definition of “success” at a job, and the goal of hiring decisions is to predict “success”
 - The only information that the hiring committee has is an exam the applicants take
 - **Proxy measurement** for ability to succeed
 - We'll assume that all applicants are from two groups: Group A (1/3 of applicants are in this group) and Group B (2/3 of applicants are in this group)

Fairness in ML

- Notation:
- X : input about a person for prediction
 - Here, the exam score
- G : variable indicating which group each person is in
 - $G = A$ or $G = B$
- Y : the “true label”
 - $Y = +1$ if a person would truly be a success, $Y = -1$ otherwise
- $\hat{Y}$: prediction for Y using an ML model

Fairness through unawareness

- First idea: prevent the ML model from knowing if applicant is in Group A or Group B
- This doesn't work in practice
 - Historical, measurement bias
 - Hidden attributes can be represented by other related attributes

Statistical Parity

- Idea: make hiring decisions equivalent across groups
- $P(\hat{Y} = +1|G = A) = P(\hat{Y} = +1|G = B)$
- If 33% of the applicants are in group B, then 33% of hired applicants should be in group B
- Pros
 - Fits some legal definitions of equity
- Cons
 - Allows for potentially unwanted strategies
 - Example: Random selection

Equal Opportunity

- Idea: true positive rate should be equivalent across groups
- $P(\hat{Y} = +1|G = A, Y = +1) = P(\hat{Y} = +1|G = B, Y = +1)$
- Pros
 - True outcome is included in definition
- Cons
 - Can be more complex to explain
 - Only considers positive outcome

Predictive Equality

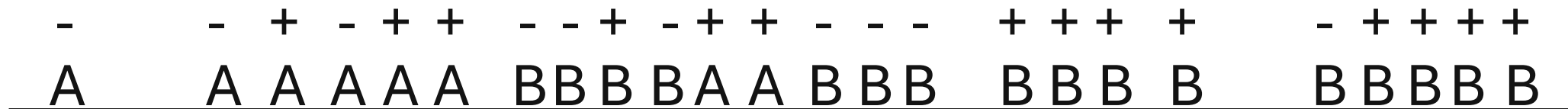
- Idea: true negative rate should be equivalent across groups
- $P(\hat{Y} = -1|G = A, Y = -1) = P(\hat{Y} = -1|G = B, Y = -1)$
- Same idea as equal opportunity
- Controls for true negatives instead
 - May be more useful when false positives are more important than false negatives

And Many more!

- The reading for Assignment 3 goes through many more definitions of fairness
- What if we wanted to satisfy Statistical Parity, Equal Opportunity, and Predictive Equality simultaneously?
 - In general, this cannot be done unless groups have the exact same underlying distribution
 - Rarely happens in the real world (historical bias, etc)

Accuracy and Fairness

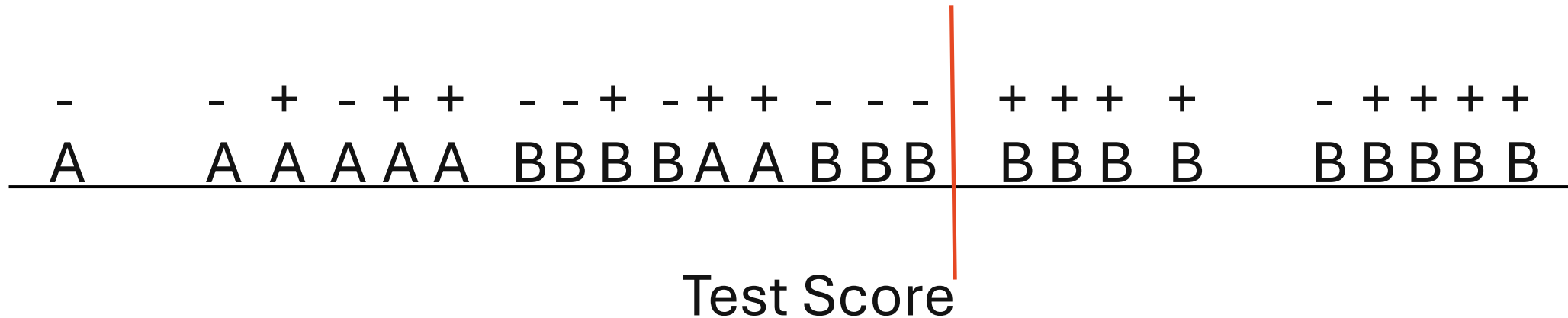
- Let's say that the testing score for Group B (2/3 of the applicants) tends to be higher than the testing score for Group A
 - Could be from systemic barriers, access to time and money for practice tests, etc
- Additionally, even though we see statistical differences between Group A and Group B, the rate at which they are actually successful in the job is the same



Test Score

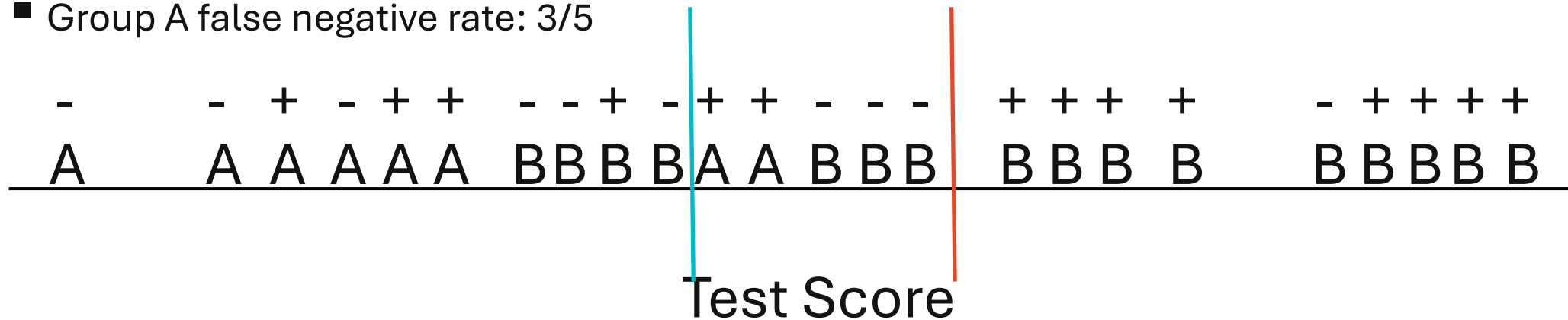
Accuracy and Fairness

- Consider a simple threshold classifier (linear classifier with 1 input, test score)
- We can just draw a line, above which applicants should be hired
- The most accurate model is not necessarily the most fair!
- Accuracy: 17/24
- Group B false negative rate: 1/9
- Group A false negative rate: 5/5



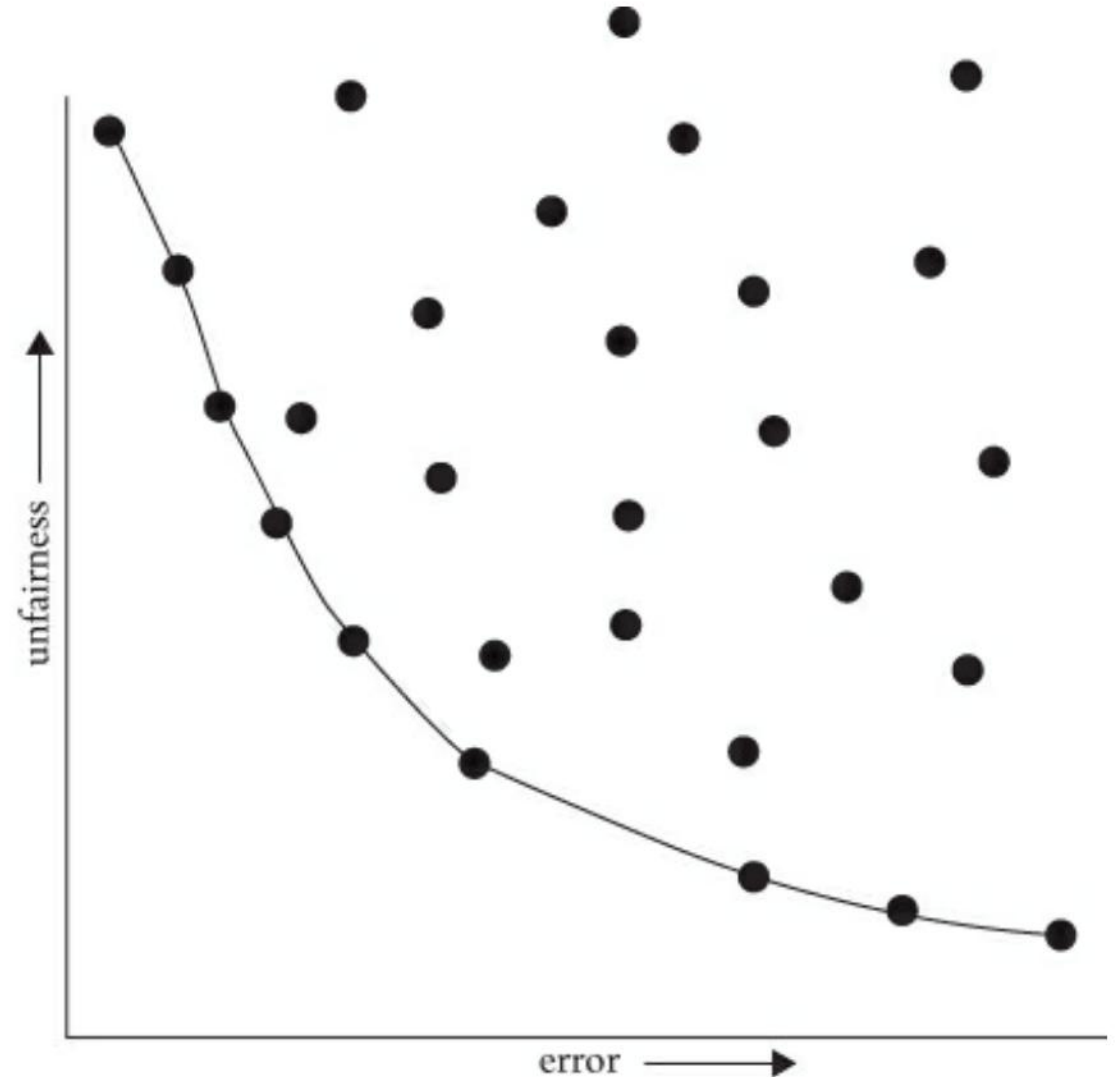
Accuracy and Fairness

- Consider a simple threshold classifier (linear classifier with 1 input, test score)
- There is often a tradeoff between accurate and fair models
 - Red line: more accurate, less fair
 - Blue line: less accurate, more fair
 - Accuracy: 16/24 (as opposed to 17/24)
 - Group B false negative rate: 1/9
 - Group A false negative rate: 3/5



Accuracy and Fairness

- Definition: Pareto Frontier
- Error: 1-accuracy
- Unfairness: $|\text{Group A FNR} - \text{Group B FNR}|$
- Visualizes tradeoff between fairness and accuracy
- Ideal model is at origin
 - No error, no unfairness



Break

Get some water, stretch your legs, chat, be back in 10

Discussion: applications and fairness

- Can fairness/discrimination in ML models be solved algorithmically? If not, what else needs to happen?
- How would What You See is What You Get and We're All Equal worldviews approach the tradeoff between accuracy and fairness?
- How would you define fairness (mathematically, or in words)?
- Do you think there is a good method for preventing (or reducing) bias in datasets?
 - If so, what is it? If not, why not?
- How might fairness considerations differ across applications?

Discussion: deployment and fairness

- Who is responsible when an algorithm makes a harmful decision — the developer, the user, or the system designer?
- Should corporations be allowed to self-regulate their AI ethics programs?
- When, if ever, is it ethical to deploy a model known to have bias if the alternative is no model at all?
- How should society define success for ethical AI — accuracy, fairness, or something else entirely?